

Testing software without requirements: using development artifacts to develop test cases

Albert Elcock · Phillip A. Laplante

Received: 17 April 2006 / Accepted: 1 August 2006 / Published online: 11 October 2006
© Springer-Verlag London Limited 2006

Abstract SourceForge, the world's largest development and download repository of open-source code and applications, hosts more than 114,500 projects with 1.3 million registered users at this writing. These applications are downloaded and used by millions across the globe. However, there is little evidence that any of these applications have been tested against a set of rigorous requirements and indeed, for most of these applications, it is likely that no requirements specification exists. How then, can one test this software? In this work an approach is presented to create a set of behavioral requirements to be used to develop test cases. These behavioral requirements were developed for a small number of projects derived from SourceForge using artifacts such as help files and software trails such as open and closed bug reports. These artifacts highlight implemented product features as well as expected behavior, which form the basis for what we term *behavioral requirements*. The behavioral requirements were then used to create test cases, which were exercised against the software. The contribution of this work is to demonstrate a viable technique for testing software when traditional requirements are unavailable or incomplete.

1 Introduction

Is a comprehensive software requirements specification document necessary for the testing of software? Consider Graham's [4] admonition: "A tester's job is to

adjudicate between what a system does and what it should do. ... A test comprises the test inputs, preconditions, and expected outcomes. ... A requirement specification is this test basis, or oracle. So, yes, testers do need requirements; otherwise, you could argue that it is not really a test." We agree with Graham.

Correct behavior is impossible to validate if it is not accurately articulated. Consider, for example, the case where an application displays a drop-down list when a menu selection is made. Is this the intended response? Can it be assumed that because most menu selections result in a drop-down list that this is the correct behavior?

Another question must be contemplated – when there is a discrepancy between a requirement and the behavior of an application, is the requirement accurate or is the implementation correct? This incongruity can only be reconciled by the expectations of the intended customer or end user of the application.

This work is based on two claims: (1) that written requirements are a prerequisite for effective behavioral testing, and (2) that SourceForge projects are largely devoid of requirements.

Behavioral testing typically refers to the process of validating a product based on a set of written requirements. However, in our framework, behavioral testing refers to the process of validating that a product functions or behaves as intended by those who developed it. Behavioral testing's focus is on functional requirements as they pertain to externally visible behavior in a form understandable by users. This definition assumes the absence of written requirements and implies the need to rebuild the intent of the developers. As a result, the approach discussed here is limited to projects that have implemented a user interface or for which a user

A. Elcock · P. A. Laplante (✉)
Engineering Division, The Pennsylvania State University,
Geat Valley School of Graduate Professional Studies,
30 East Swedesford Road, Malvern, PA 19355, USA
e-mail: aelcock@motorola.com; plaplante@psu.edu

interface may be designed (e.g., a library that provides a file system interface).

The use of the term requirements in the second claim, that SourceForge projects are largely devoid of requirements, requires amplification. Requirements, in this context, refer to a written collection of statements specifically identified and organized in one or more locations that define the expected behavior of the associated software. Projects from the SourceForge repository were selected for this study because it was necessary to use software for which few or no written requirements exist. It is true that many, if not all, SourceForge projects have defining statements that can be assembled into one or more requirements documents. However, it is generally accepted that SourceForge projects were developed without written requirements [2].

The idea of an approach to testing when requirements are unavailable is not novel. Though it was not the focal point of her paper, in the absence of requirements Graham suggests interviewing users who are familiar with the system under test, consulting user manuals and using intuition. Also, in the context of reconstructing the evolution of a software system, German coined the term *software trails*, referring [3] to “information left behind by the contributors to the development process of the product, such as mailing lists, web sites, version control logs, software releases, documentation, and the source code.” In our approach testing without requirements consists of using information collected through vehicles identified by Graham and German (hereinafter referred to as *development artifacts*). Our approach essentially combines these two approaches to provide a useful framework for testing.

The remainder of this paper is organized as follows. Section 2 defines the method and process (also referred to as the *methodology*) for developing test cases and reconciling discrepancies based on development artifacts. In Sect. 3, the application of this methodology to a small sample of SourceForge projects as well as relevant results is reported. The conclusion follows an explanation, in Sect. 4, of how discrepancies and process errors were resolved, and discusses lessons learned.

2 Creating test cases from development artifacts

Given an application that was developed without requirements test cases can be developed using existing development artifacts. The method consists of five steps:

1. Assemble development artifacts
2. Create behavioral specification

3. Develop test cases
4. Execute test cases
5. Reconcile discrepancies

We further explain these steps in the following sections.

2.1 Assemble development artifacts

Assembling development artifacts begins the process of extracting the information necessary for developing behavioral requirements. Of primary importance, as would be expected, is any information that pertains to the behavior of the application. Development artifacts may include any of the following:

- User manuals or help files
- Release notes
- Bug reports and support requests
- Application forums
- The application itself
- Other ephemera that have been posted to the application site or are otherwise available on the web.

Many SourceForge projects are without several of these development artifacts. However, the methodology described subsequently can be applied with or without the existence of each artifact.

2.2 Create behavioral specification

As mentioned earlier, it is necessary to have clear requirements in order to develop and execute test cases. This step in the process fulfills the need for requirements using the artifacts that were assembled in the previous step.

Once the development artifacts are collected and analyzed, a behavioral specification can then be created. This specification should be written as unambiguously as possible. The goal during this step of the process is to use the development artifacts to document the behavior of the software as intended by the developers.

2.2.1 User manuals and help information

Starting with user manuals, statements about the behavior of an application in response to user input can be directly related to the behavioral specification. User manuals are particularly well suited for this purpose since describing the response of software to user stimulus is germane to their existence.

Help information, such as support web sites and application help menus are also rich in information that can

either be used directly or abstracted to define behavioral requirements.

2.2.2 Release notes

Release notes are typically of limited benefit to the process of developing test cases since they tend to focus on describing which features are implemented in a given release. There is usually no description of how those supported features should function. However, release notes are particularly important in resolving the conflict that arises when an application does not respond as expected for features that were partially implemented in or removed from future implementation.

2.2.3 Bug reports and support requests

Bug reports can be a great source for extracting behavioral responses because they are typically written by users, they identify troublesome areas of the software and they often clarify a developer's intention for the particular behavior. It is true that in some cases bug reports contain too much implementation detail to be useful for extracting behavioral responses, but they can be discarded if behavior cannot be extrapolated from the details.

In many ways, the content of support requests is similar to bug reports in that they identify unexpected behavior. Unlike bug reports, however, support requests can sometimes be helpful in identifying features that are not fully implemented as well as providing information that illuminates the expectations of users. It is important to investigate both since, in addition to providing the insights mentioned, they may also aid in rationalizing unexpected behavior.

2.2.4 Application forums

There is a "Forum" web link associated with many SourceForge projects. When selected, an additional web link, "Open Discussion", can be accessed to retrieve what the title implies. Ranging from useless to relevant, open-discussion postings need to be carefully filtered and applied only when other development artifacts are lacking. As with other artifacts, these postings can also be used to clarify behavior.

2.2.5 Software under test

As a last resort in the absence of any documentation, the software being tested could itself be an input to developing structural (glass-box) test cases. Assuming that this approach is necessary, the reality is that it will really be

the tester's characterization of correct behavior that will largely prevail in defining the test cases.

2.3 Develop test cases

Once the behavioral specification has been completed, it is a relatively simple task to develop test cases. Test cases should only exist as an output of the behavioral specification. Depending upon the available development artifacts, it should be expected that details used in the test cases may be directly extracted from them. Nevertheless, the test cases should remain consistent with the associated behavioral requirement.

The components of a test case of particular interest include: inputs, preconditions, expected behavior, and actual behavior. It is important to understand both the expected and actual behavior since test execution will be based on reconstructed requirements.

2.4 Execute test cases

The successful execution of test cases in this framework indicates the confluence of process steps that signify a successful methodology. Test cases that result in a failure represent a discrepancy that must be reconciled as further confirmation of the viability of this methodology.

Completing this process is a first step toward validating the reconstructed requirements, test cases and the application. The success of test execution is based on the consistency of the development artifacts with the implementation and the accuracy with which the details in the development artifacts were transcribed.

2.5 Reconcile discrepancies

Invariably a number of test cases will fail, resulting in the need to make a judgment about where culpability lies. This is the most difficult step in the process as it requires a reliable understanding of the intent of the application developers or the expectations of users.

Before discussing an approach to reconciling discrepancies, a definition of discrepancy should be provided. For the purposes of this work, a discrepancy is limited to any mismatch between the actual behavior of an application and the expected behavior written in the test case. When a discrepancy occurs, it must be determined whether the test case and/or the behavioral requirement is written incorrectly or the application behaved inappropriately. To accomplish this reconciliation, it is necessary to retrace the process steps in reverse order of execution:

- Re-execute the test case to confirm that a discrepancy does exist
- Compare the test case with the behavioral specification for consistency
- Compare the behavioral specification with the development artifact(s) from which the requirement was derived

After retracing these steps, if a discrepancy still exists, then more than one development artifact should be used to verify the accuracy of the interpretation in the behavioral specification. If the accuracy cannot be verified (e.g., there is only one statement on the subject in a development artifact or there is only one development artifact), then it is necessary to record the test case as a failure.

3 Testing the hypothesis

With the methodology outlined, it was then necessary to select one or more SourceForge projects with which the hypothesis could be tested. It was determined that there would be a rigorous adherence to the process steps outlined earlier with any deviations documented. In this section, the selection process will be discussed as well as the necessary deviations from the defined process. Later, statistics will be presented along with an explanation of how deviations were addressed.

3.1 Project selection

As a first step to applying the methodology outlined so far, it was decided that four SourceForge projects would be selected. Only projects from the 40 all-time top downloads were considered because they represent projects of significance to many users. Projects in this category were downloaded 5–197 million times since their creation (see Table 1 for a selected sample).

Table 1 Sample of top SourceForge downloads

Project name	Downloads	Age (years)
eMule	197,328,840	4
Azureus	117,866,142	3
BitTorrent	51,850,559	5
NASA World Wind	8,536,779	4
aMSN	7,936,208	4
JBoss.org	7,460,211	5
WinSCP	7,315,293	3
AC3Filter	7,113,179	4
VisualBoyAdvance	5,829,681	4
PDFCreator	5,390,339	4

The age of the application, based on the time it was registered on SourceForge, was considered in the event that a correlation could be established between the age and the maturity of the development artifacts. No correlation was found; therefore, age-related information was not considered any further in this analysis.

Each of the projects in the 40 all-time top downloads was further reviewed to isolate potential candidates for this work. To be included in this study, the projects had to make available an executable for Microsoft Windows XP, be sufficiently dissimilar in function and objective, provide a graphical user interface and have one or more development artifacts. It was observed that all projects in the top-40 category had at least one development artifact. In many cases, such as eMule and AC3Filter, all of the development artifacts were hosted on application-specific web sites not on SourceForge.

The four projects chosen along with their topic, intended audience and relevant development artifacts acquired through, or linked to by, SourceForge are shown in Table 2. All four were used to lend credibility to the existence of development artifacts, but the complete methodology was only applied to the NASA World Wind and PDFCreator projects.

After the projects were chosen, each was downloaded to ensure it could be executed on a Microsoft XP workstation. In addition, a short time was spent becoming familiar with each application, but care was taken to avoid developing an intimate knowledge of their workings. It was important to remain unbiased by the implemented behavior as the primary inputs to the specification process include other development artifacts.

3.2 Applying the methodology

The process of assembling development artifacts began after it was confirmed that each application would execute on a Microsoft XP workstation and while becoming familiar with the application. The first artifact collected was extracted from the “Help” option of the application itself. This was an unplanned discovery and though it does not denote a deviation from the defined methodology, it is worthy of mention. All of the applications selected provided access to help information directly from the application. In some cases, the help information was presented in the form of a user manual, and in others, multiple web links were provided. This was not one of the selection criteria, but is rather an observation noted after the selection process concluded.

Once the existence of development artifacts was established, 20 requirement statements primarily based on features pertaining to the existence of the software

Table 2 Research project descriptions

Project name	Project version	Topic	Intended audience	Development artifacts
PDFCreator	0.9.0	Office suites, printing	End users/desktop, system administrators	• Help information • User manual • Release notes • Bug reports • Support requests • Forums
NASA World Wind	1.3.3	Education, viewers, GIS, visualization	End users/desktop, education, science/research	• Help information • Bug reports • Support requests
AC3Filter	0.70b	Sound/audio	End users/desktop	• Help information • Bug reports • Support requests • Forums
eMule	0.47a	File sharing	End users/desktop	• Help information forums

were developed for the two selected applications. The original statement quoted from the development artifact, its source and any requirements generated, were documented. A partial sample from the NASA World Wind application is shown in Table 3.

For ambiguous requirements such as the following, “NASA World Wind should change the terrain elevation of the Earth if the ‘1’ key is pressed”, other resources were searched to resolve the ambiguity. If clarifying information could not be found (as in this case), then the test case was constructed based on the statement as written.

Test cases were developed based on the requirement statements that were generated. Each of the test cases followed the format shown in the example in Table 4. If a source other than the behavioral statement was used to develop the test case, then it was listed with the sources. Each test case was constructed using the inputs, preconditions, and expected behavior extracted from the development artifacts for each project. The actual behavior is a record of the results when the test case was executed against the particular application. As would be expected, the response of the software was not recorded as the “actual behavior” until the test case was executed.

Once the test case was executed, its disposition with all observations were recorded as the “actual behavior” as shown in Table 5. The three possible test case dispositions included: “PASSED”, “FAILED” and “could not be completed”. As previously mentioned, any test case for which the actual behavior did not match the expected behavior is counted as a discrepancy. Therefore, all FAILED test cases were automatically noted as discrepancies.

A discrepancy is only considered when the actual response of the software does not match the expected response delineated in the test case, as explained earlier. When this occurs, the reconciliation process outlined below was followed.

Re-execute the test case to confirm that a discrepancy does exist.

Assume the test case was written incorrectly. Once it was confirmed that a discrepancy does exist, the first step was to reconcile the test case with the behavioral statement it references.

Assume the behavioral requirement was written inaccurately. If the test case and behavioral statement were consistent with each other, then it was assumed that the behavioral statement was written inaccurately. Development artifacts were then consulted to either verify the accuracy of the behavioral requirement or cause a change to the requirement.

Assume the application has a bug. If consistency could be confirmed between the behavioral statement and the associated development artifacts, then it was safely assumed to be a bug and the disposition of the test case remained unchanged.

Test cases were categorized as “could not be completed” if, for any reason, the test case could not be executed as written. This disposition was primarily a result of missed test steps, although in some cases, it was because the relevant menu selection could not be found.

4 Analyzing the results

It was during execution of the test cases that the effectiveness of the methodology could be confirmed. Only

Table 3 Behavioral requirement sample

Original statement	Source	Behavioral requirement
To begin moving around try left-drag-clicking and moving the mouse. This pans around the Earth while keeping you at the same distance. You can also try single-left-clicking and letting go at any point on the Earth. World Wind will automatically move you to that spot. (You can use the arrow keys on your keyboard instead of a mouse)	Web manual – introduction	NASA World Wind shall rotate the Earth to the left if the left mouse button is held down and the mouse moved to the left
To rotate, try right-drag-clicking and moving the mouse. This will spin the perspective around.		NASA World Wind shall rotate the Earth to the right if the left mouse button is held down and the mouse moved to the right
Please note, single-right-clicking does not work in this mode. (You can use the W, A, S, D keys on the keyboard instead of a mouse)		NASA World Wind shall stop rotating the Earth when the left mouse button is released
		NASA World Wind shall rotate the Earth to the location selected if the left mouse is depressed and released on any point of the Earth
Terrain elevation can be adjusted by pressing any number key from “1”–“9”	Web manual – key chart	NASA World Wind shall rotate the Earth to the left if the left arrow key is depressed
		NASA World Wind shall stop rotating the Earth when either arrow key is released
Pressing “0” will flatten the terrain		NASA World Wind shall change the terrain elevation of the Earth if the “1” key is pressed
		NASA World Wind shall change the terrain elevation of the Earth if the “9” key is pressed
		NASA World Wind shall flatten the terrain elevation of the Earth if the “0” key is pressed

Table 4 Sample test case without result

Test case	NASA-TC-1
Inputs	Press and hold the left mouse button on the upper left quadrant of the Earth Move the mouse to the left
Preconditions	A model of the Earth is displayed
Expected behavior	NASA World Wind should rotate the Earth to the left until the mouse stops moving
Actual behavior	
Sources	NASA-1

after the test cases were executed could attempts be made to reconcile any discrepancies found and identify process errors. An account of the method is given in this section along with lessons learned.

4.1 Resolving discrepancies

As previously stated, a test case disposition of “FAILED” was counted as a discrepancy. A discrepancy could occur for one of the following three reasons:

1. The test case was either incorrectly constructed or incorrectly executed.
2. The behavioral requirement was written inaccurately.
3. The software has a defect.

Of the three reasons for a discrepancy, the first two are indicative of a process error. Therefore, when a discrepancy occurred it was necessary to determine the root cause to eliminate process errors and correctly identify failures of the software to behave as expected.

As shown in Table 6, 45% of the total test cases executed without modification (i.e. first pass) resulted in a

Table 5 Sample test case with result

Test case	NASA-TC-1
Inputs	Press and hold the left mouse button on the upper left quadrant of the Earth Move the mouse to the left
Preconditions	A model of the Earth is displayed
Expected behavior	NASA World Wind should rotate the Earth to the left until the mouse stops moving
Actual behavior	FAILED – NASA World Wind rotated the Earth to the left as expected, but the Earth rotated unpredictably after the left mouse button was released and the mouse was still held
Sources	NASA-1

“FAILED” disposition (Note: the test cases that were assigned a disposition of “could not be completed” were not included in this number). It should come as no surprise that the PDFCreator application, which provided much more detailed documentation than NASA World Wind, had comparatively fewer discrepancies.

To resolve the discrepancies, each of the relevant test cases was compared first against the behavioral requirement for consistency. Once determined to be consistent, the behavioral requirement was confirmed to be in harmony with the development artifact. In the case of the PDFCreator software, the test case was also compared with development artifacts because of the level of detail available. Missing test steps were discovered in one case and a misread artifact in the other. Therefore, the disposition of both discrepancies was reclassified to “PASSED”.

By contrast, one of the NASA discrepancies was indisputably a failure and was noted as such. However, it could be argued that the 11 others, all of which pointed to a common root cause, were not strictly failures. The particular behavior occurred when the mouse or arrow keys were used to navigate the Earth. In these cases, the Earth continued to rotate even after the mouse button or the arrow key was released. There was some ambiguity in the statements taken from the development artifacts upon which the behavioral requirements were written and, although several other artifacts were consulted, no

clarification of intent could be found. Nevertheless, it is inconceivable that the developers intended the software to function as it does and thus the disposition of these test cases remained unchanged.

4.2 Process errors

Twenty-two and a half (22.5) percent of the test cases executed resulted with a “could not be completed” disposition, as shown in Table 7. The classification of a test case as “could not be completed” is suggestive only of a process problem. Curiously, all of these test cases were associated with the PDFCreator application. However, this became less a surprise after it was recognized that the PDFCreator test cases were much more detailed than those for NASA World Wind because of the maturity of the former’s development artifacts.

The “could not be completed” classification initiated a comparison of the test case first with the associated behavioral requirement. In all cases it was necessary to consult additional development artifacts to determine the root cause. The causes, shown in Table 8, include the following:

Test steps that were simply overlooked in the initial design of the test case. For five of the test cases, this was easily substantiated using the development artifacts. The missing test steps for the other three test cases were discovered under a menu selection other than that implied by the development artifacts. Further study of additional artifacts yielded no new information; therefore, the three test cases were updated to include the missing test steps. Each was then reclassified as “PASSED” because the software functioned as defined.

The option being tested was not installed. The final test case that could not be completed was a result of not installing the language option that was being tested. This

Table 6 First pass statistics: – number of discrepancies

Project name	Number of discrepancies	% of test cases
PDFCreator	2	18.2
NASA World Wind	12	60
Total	14	45.2

Table 7 First pass statistics – number of modified test cases

Project name	Number of modified test cases	% of test cases
PDFCreator	9	45
NASA World Wind	0	0
Total	9	22.5

Table 8 Causes of “could not be completed” disposition

Number of test “could not be completed” cases	% of test “could not be completed” cases	Cause
8	88.9	Missing test steps
1	11.1	Option not installed

problem was discovered during the review of the associated development artifacts. The test case was neither reclassified nor re-executed because it was decided that the language option would not be installed to enable this feature.

Interestingly enough, the ambiguity related to the NASA World Wind discrepancies notwithstanding, none of the behavioral requirements for either application needed to be updated. It should not be presumed though that this is representative of the norm and it may simply be a result of either the small number of samples developed or the level of abstraction of the behavioral requirements.

4.3 Lessons learned

A brief analysis of the results reviewed above underscores three observations about the level of detail available in the development artifacts:

The level of detail is inversely proportional to the number of discrepancies. This fact, of course, makes sense. After all, the development artifacts record much of the intent of the application developers and the desires of its users. The availability of these details reduces the need to interject intuition or make assumptions about the correctness of particular behaviors.

There is a strong correlation between the level of detail and the number of process errors. Although this phenomenon initially came as a surprise, it was easily rationalized after the root causes were determined. Unlike its effect on the number of discrepancies, having a greater level of detail resulted in more mismatches in the test steps delineated. As noted earlier, this deficiency was relatively easy to correct.

Regardless of its level of detail, there may be a need to consult the implementation itself. As stated, a menu referred to in a development artifact was found as a result of searching the application. This step was necessary because the exact location was not specified, but the behavior of the application when the menu option is selected was clearly delineated.

Another lesson learned is that, without domain knowledge, it is possible to apply the methodology outlined in this paper. However, the need for domain knowledge should be emphasized since, for example in testing NASA World Wind, a response of the software could be evaluated, but its correctness could not. For instance, one of its development artifacts explains that right-click dragging will cause the view of the Earth to be slanted in the direction in which the mouse is moved. Another artifact recommends that, when the view of the Earth is slanted, the cursor should behave at all altitudes as it does at sea level [1]. This discrepancy exposes a detail,

undetected during the application of this methodology, which could only be articulated by someone with an understanding of the domain.

In addition, this methodology could also be applied to enterprise applications for which there are minimal or no requirements. Software analysis tools provide metrics such as feature points. Using feature points, an estimate of the total number of behavioral requirements could be developed and used to project the number of test cases that would be necessary. Regardless of the type or scope of a project, it has already been shown that written requirements are a necessary input to test. Additional works that focus on requirements discovered during the test process could also be applied in this context [5]. Therefore, armed with these tools, the test planning process, including the generation of behavioral requirements, could commence.

5 Conclusion

In this work a methodology for developing test cases based on development artifacts was defined and demonstrated. After establishing the need for written requirements as a prerequisite to the effective development of test cases, the methodology was outlined to include the generation of behavioral requirements. A small sample of SourceForge projects was then used to demonstrate the efficacy of the defined methodology.

It was shown that a strong correlation could be made between the level of detail in the development artifacts and the number of discrepancies and process errors encountered when applying this methodology. As previously highlighted, the level of detail available is inversely proportional to the number of discrepancies and directly correlated with the number of process errors. This result is significant because the former result raises questions about satisfying the needs and expectations of users, and provides a strong case for written requirements. The latter result is profitable for maintaining the accuracy and clarity of the development artifacts.

In addition, the case was made for a broader application of this methodology to enterprise software with the assumption that requirements were not readily available. In combination with the aforementioned tools and methods, a very compelling solution to developing test cases can be employed. Moreover, behaviorally constructed test cases can be used as a basis for future regression testing of the software.

A final assertion that must be made is that constructing test cases as discussed is not only possible but also effective. It is true that claims about the efficacy of the methodology described in this paper should be tem-

pered with a reminder of the size of the sample used in its confirmation. However, the claim can confidently be made that a framework for further studies has been solidly established.

References

1. Bulletin (2006) World wind forum. <http://forum.worldwind-central.com>. Cited 23 May 2006
2. Crows ton K, Skuzzy B, (2005) Exploring the strengths and limits of open source software engineering processes: a research agenda. In: Second workshop on open source software engineering, Orlando, Florida, May 25, 2002, pp 1–2
3. German D (2004) Using software trails to reconstruct the evolution of software. *J Softw Maint Evol Res Practice* 16:367–384
4. Graham D (2002) Requirements and testing: seven missing-link myths. *IEEE Softw*, Sep/Oct:15–17
5. Lutz R, Mikulski I, (2003) Resolving requirements discovery in testing and operations. In: *Proceedings of the 11th IEEE international requirements engineering conference*, pp 33–41